



DIPARTIMENTO DI ELETTRONICA,
INFORMAZIONE E BIOINGEGNERIA

Politecnico di Milano

Machine Learning (Code: 097683)

February 16, 2024

Surname:

Name:

Student ID:

Row:

Column:

Time: 2 hours

Maximum Marks: 33

- The following exam is composed of **8 exercises** (one per page). The first page needs to be filled with your **name, surname and student ID**. The following pages should be used **only in the large squares** present on each page. Any solution provided either outside these spaces or **without a motivation** will not be considered for the final mark.
- During this exam you are **not allowed to use electronic devices**, such as laptops, smartphones, tablets and/or similar. As well, you are not allowed to bring with you any kind of note, book, written scheme, and/or similar. You are also not allowed to communicate with other students during the exam.
- The first reported violation of the above-mentioned rules will be annotated on the exam and will be considered for the final mark decision. The second reported violation of the above-mentioned rules will imply the immediate expulsion of the student from the exam room and the **annulment of the exam**.
- You are allowed to write the exam either with a pen (black or blue) or a pencil. It is your responsibility to provide a readable solution. We will not be held accountable for accidental partial or total cancellation of the exam.
- The exam can be written either in **English** or **Italian**.
- You are allowed to withdraw from the exam at any time without any penalty. You are allowed to leave the room not early than half the time of the duration of the exam. You are not allowed to keep the text of the exam with you while leaving the room.
- **Three of the points will be given on the basis on how quick you are in solving the exam.** If you finish earlier than 30 min before the end of the exam you will get 3 points, if you finish earlier than 20 min you will get 2 points and if you finish earlier than 10 min you will get 1 point (the points cannot be accumulated).
- The box on Page 10 can only be used to complete the Exercises 7, and/or 8.

Ex. 1	Ex. 2	Ex. 3	Ex. 4	Ex. 5	Ex. 6	Ex. 7	Ex. 8	Time	Tot.
/ 7	/ 7	/ 2	/ 2	/ 2	/ 2	/ 4	/ 4	/ 3	/ 33

Exercise 1 (7 marks)

Write the pseudocode of the perceptron algorithm and prove that the update rule decreases the error for the currently processed sample at each iteration.

Exercise 2 (7 marks)

Provide an overview of the **feature selection** methods that you know and explain their pros and cons.

Exercise 3 (2 marks)

Consider the following snippet of code:

```

1 Q = np.zeros(nS * nA)
2 Q_old = np.ones(nS * nA)
3 V = np.zeros(nS)
4
5 while np.any(Q != Q_old):
6     Q_old = Q
7     for s in range(nS):
8         V[s] = np.max(Q_old[s * nA : (s + 1) * nA])
9     Q = R_sa + gamma * P_sas @ V

```

1. What algorithm is the snippet of code above implementing? What problem is it trying to solve?
2. Is code snippet above correct? If not, list the mistakes and provide a fix for each of them.
3. Is the **while** loop guaranteed to stop after a finite number of iterations (if $\text{gamma} < 1$)? If yes, provide an upper bound on the maximum number of iterations. If not, provide a counterexample.

1. The snippet is implementing the Value Iteration algorithm. The algorithm is trying to solve the control problem in reinforcement learning (actually, planning).
2. The snippet is correct. One may argue that the loop control condition at line 5 is incorrect since it does not guarantee termination and should be replaced with, for instance, `np.any(np.abs(Q - Q_old) > eps)`, where `eps > 0` is a user-defined threshold.
3. Value Iteration is guaranteed to converge only asymptotically. Thus, the **while** with the loop control condition at line 5 is not guaranteed to terminate after a finite number of steps (of course, one may argue that the loop can terminate anyway because of the finite machine precision). A simple counterexample is given by $nS=nA=1$, $R_{sa} = 1$. Here, at the i th iteration, we have that $Q_{old} = \sum_{\ell=0}^{i-1} \gamma^\ell$ and $Q = \sum_{\ell=0}^i \gamma^\ell = Q_{old} + \gamma^i$ and, consequently, $Q \neq Q_{old}$ over all iterations.

Exercise 4 (2 marks)

Tell if the following statement holds or not for the logistic regression and KNN methods. Provide motivations for your answers.

1. Exists a closed form solution for the optimization procedure required to train the model.
2. They are methods used for classification, but can be easily adapted to solve regression problems.
3. The prediction phase for the model is computationally light.
4. They allow the use of regularization methods for model selection purposes.

1. FALSE for both: logistic regression is trained using an iterative procedure (is guaranteed to converge but there is not closed form solution), KNN is a non-parametric method that do not require any training process.
2. FALSE for logistic regression, TRUE for KNN: logistic regression represents an extension of the linear regression technique for classification problems. KNN can be used also for regression, averaging the output of the selected samples instead of applying a majority voting approach at the end of the procedure.
3. TRUE logistic regression, DEPENDS KNN: logistic regression requires a number of operations for prediction which is linear in the dimension of the parameter vector, KNN requires to compute all the distance between the samples in the training set and the new sample (linear in the sample number). If we assume to have a small number of samples, then this operation can be performed in a fast way too.
4. TRUE for both: the logistic regression loss function can be modified to include penalties for regularization purposes, the KNN method uses the parameter K as a regularization tool (large K implies large regularization).

Exercise 5 (2 marks)

Tell whether the following statements are true or false. Justify your answers.

1. SARSA is a prediction method since it makes use of the Bellman expectation equation for updating the value function.
2. Q-learning is based on Temporal Difference learning while SARSA on Monte Carlo estimation.
3. Both SARSA and Q-learning necessitate to update the policy used to collect samples with the ϵ -greedy policy improvement.
4. Differently from SARSA, Q-learning does not need to wait for the action played in the next state for updating the value function.

1. FALSE. Although it uses the Bellman expectation equation, SARSA is a control method since it aims at learning the optimal Q-function Q^* as well as Q-learning.
2. FALSE. They are both based on Temporal Difference learning since they *bootstrap* the previous estimate of the Q-function for defining the targets of the update.
3. FALSE. This is true for SARSA only, where the policy used to collect samples needs to converge to the optimal policy, being SARSA an on-policy method. Being Q-learning an off-policy method, the policy used to collect samples does not need to converge to the optimal policy. Thus, for the latter, the ϵ -greedy policy improvement is not needed. (One may argue that other improvements are possible for SARSA other than the ϵ -greedy policy improvement. What matter is that those converge to the optimal policy.)
4. TRUE. SARSA needs such an action a_{t+1} for defining the TD target that contains $Q(s_{t+1}, a_{t+1})$. Instead, Q-learning uses in the TD target the term $\max_{a' \in \mathcal{A}} Q(s_{t+1}, a')$; thus, it does not need to wait for a_{t+1} .

Exercise 6 (2 marks)

Consider a problem of designing an autonomous driving agent for a car. The system's performance must be evaluated based on both the time it takes to reach a designated location and the safety of the driver and any individuals the car may come across along its path. The system should replace the driver in the possible decision/actions they could perform in a car with automatic gearbox.

1. Model the problem as an MDP and provide a technique to learn the best driving policy. Motivate your decisions.
2. In the case the learning process is performed on a real car in a real-world setting, would you prefer to adopt an off-policy or an on-policy approach? (In the case it is off-policy, specify also the policy the agent should follow)

1. Since we want to find an optimal strategy we want to solve a control problem. The MDP modeling such a problem has the following elements:

- States: location, speed, acceleration of the car. We might add also other parameters related to the distances from possible obstacles.
- Actions: steer (left, right with a specific angle), accelerate/decelerate (with specific intensity), emergency break activation.
- Transition matrix: stochastic and unknown, since the modelization of the possible situations is significantly large and complex to be modeled.
- Reward: weighted sum of a value corresponding to the decrease in terms of distance to the final target and an index related to the safety of the action (considering penalties if the car hits some obstacles).
- $\gamma < 1$: since there might be some trajectories which are infinitely long (loops).
- Initial state: initial state of the car (initial position, zero speed, and acceleration).

Since some of the elements are unknown, we need to resort to RL control methods, like Q-learning and SARSA.

2. Using an on-policy approach would mean that the learning would incur in a significant number of experiments in which accidents and other unwanted behaviour would occur, therefore it is not a good idea in this setting. Instead, if we have already a policy that ensures the safety of the car, the passengers and people around the car, as long as it is still exploring enough, we can follow that policy and learn in an off-policy manner, for instance using Q-Learning.

Exercise 7 (4 marks)

Consider an initial parameter $\mathbf{w}^{(0)} = [1 \ 2 \ 0]^\top$ and a loss function of the form:

$$J(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^N (\mathbf{w}^\top \mathbf{x}_n - t_n)^2 + \frac{\lambda}{2} \mathbf{w}^\top \mathbf{w}.$$

1. Derive the update given from the gradient descent for a generic input;
2. Apply the formula to the input $\mathbf{x}_1 = [2 \ 1 \ 1]^\top$, $t_1 = 2$, with a learning rate $\alpha = 0.3$ and a regularization coefficient $\lambda = 0.5$;
3. Let us assume to iteratively run such a procedure over the entire dataset. Do we have any convergence assurance of this procedure?

1. The update for a given parameter for the gradient descent for a single input is:

$$w_j^{(1)} = w_j^{(0)} - \alpha[(\mathbf{x}_1^T \mathbf{w}^{(0)} - t_1)x_{1j} + \lambda w_j^{(0)}];$$

2. The update for the given sample is:

$$w_0^{(1)} = 1 - 0.3[(4 - 2) \cdot 2 + 0.5 \cdot 1] = -0.35,$$

$$w_1^{(1)} = 2 - 0.3[(4 - 2) \cdot 1 + 0.5 \cdot 2] = 1.1,$$

$$w_2^{(1)} = 0 - 0.3[(4 - 2) \cdot 1 + 0.5 \cdot 0] = -0.6,$$

and the final coefficient is $\mathbf{w} = [-0.35, \ 1.1, \ -0.6]$;

3. Since the loss function is convex, the procedure will converge to the unique minimum of the loss function. This might not be true if the learning rate α is too large.

Exercise 8 (4 marks)

Suppose we have a training set composed of $N = 100$ samples and we use it to train two binary classifiers $\hat{h}_1$ and $\hat{h}_2$ belonging to the hypothesis spaces:

- $\hat{h}_1 \in \mathcal{H}_1$, where $|\mathcal{H}_1| = e^6$;
- $\hat{h}_2 \in \mathcal{H}_2$, where $|\mathcal{H}_2| = +\infty$ and its VC-dimension is $\text{VC}(\mathcal{H}_2) = 2$.

Compute the upper bound of the classification error with confidence $1 - \delta = 1 - e^{-2}$ for the two classifiers in the following two scenarios:

- The two classifiers $\hat{h}_1$ and $\hat{h}_2$ are consistent learners.
- The two classifiers $\hat{h}_1$ and $\hat{h}_2$ have a training error of $\hat{\mathcal{L}}(\hat{h}_1) = 0.3$ and $\hat{\mathcal{L}}(\hat{h}_2) = 0.1$, respectively.

Hint. Recall the learning theory bounds:

$ \mathcal{H} < +\infty$ and consistent learning	$\mathcal{L}(\hat{h}) \leq \frac{\log \mathcal{H} + \log\left(\frac{1}{\delta}\right)}{N}$
$ \mathcal{H} < +\infty$	$\mathcal{L}(\hat{h}) \leq \hat{\mathcal{L}}(\hat{h}) + \sqrt{\frac{\log \mathcal{H} + \log\left(\frac{1}{\delta}\right)}{2N}}$
$ \mathcal{H} = +\infty$	$\mathcal{L}(\hat{h}) \leq \hat{\mathcal{L}}(\hat{h}) + \sqrt{\frac{\text{VC}(\mathcal{H}) \log\left(\frac{2eN}{\text{VC}(\mathcal{H})}\right) + \log\left(\frac{4}{\delta}\right)}{N}}$

and $\sqrt{\frac{\log(40000e^3)}{100}} \approx 0.37$.

We are going to compute the upper bound to the true error $\mathcal{L}(\hat{h})$ holding with probability $1 - \delta = 1 - e^{-2}$ and propose to deploy the classifier with smallest upper bound.

Consistent learners case. For $\hat{h}_1$, being $\mathcal{H}_1$ finite, we apply the first formula:

$$\mathcal{L}(\hat{h}_1) \leq \frac{\log |\mathcal{H}_1| + \log \left(\frac{1}{\delta}\right)}{N} = \frac{\log e^6 + \log e^2}{100} = 0.08.$$

For $\hat{h}_2$, although it is consistent, its hypothesis space $\mathcal{H}_2$ is infinite. Thus, we need to apply the third formula anyway:

$$\mathcal{L}(\hat{h}_2) \leq \underbrace{\hat{\mathcal{L}}(\hat{h})}_{=0} + \sqrt{\frac{\text{VC}(\mathcal{H}_2) \log \left(\frac{2eN}{\text{VC}(\mathcal{H}_2)}\right) + \log \left(\frac{4}{\delta}\right)}{N}} = \sqrt{\frac{2 \log(100e) + \log(4e^2)}{100}} = \sqrt{\frac{\log(40000e^3)}{100}} \approx 0.37$$

Thus, in this second case, we propose to deploy $\hat{h}_1$.

Agnostic learners case. For $\hat{h}_1$, being $\mathcal{H}_1$ finite, we apply the second formula:

$$\mathcal{L}(\hat{h}_1) \leq \hat{\mathcal{L}}(\hat{h}_1) + \sqrt{\frac{\log |\mathcal{H}_1| + \log \left(\frac{1}{\delta}\right)}{2N}} = 0.3 + \sqrt{\frac{8}{200}} = 0.3 + \sqrt{0.04} = 0.5.$$

For $\hat{h}_2$, we apply the third formula:

$$\mathcal{L}(\hat{h}_2) \leq \hat{\mathcal{L}}(\hat{h}) + \sqrt{\frac{\text{VC}(\mathcal{H}_2) \log \left(\frac{2eN}{\text{VC}(\mathcal{H}_2)}\right) + \log \left(\frac{4}{\delta}\right)}{N}} = 0.1 + \sqrt{\frac{2 \log(100e) + \log(4e^2)}{100}} \approx 0.47$$

Thus, in this second case, we propose to deploy $\hat{h}_2$.